

CPU, Memory & the Boot Process

Registers and the Stack

The CPU has a tiny set of fast storage locations called **registers** (**RAX**, **RBX**, **RIP**, **RSP**, etc. on x86-64). The **stack** is a region of memory that grows downward, used for function calls and local variables. Every function call pushes a **return address** onto the stack.

Security insight: Buffer overflow attacks overwrite the return address. This is the classic stack-smashing exploit – understanding registers and the stack is essential to understanding half of all modern exploitation.

The Memory Hierarchy

Registers	(fastest, smallest)	~1 cycle
L1 cache	~4 cycles	
L2 cache	~12 cycles	
L3 cache	~40 cycles	
RAM	~200 cycles	
SSD/NVMe	~100,000 cycles	
Disk/HDD	~5,000,000 cycles	

The gap between "fast" and "slow" memory is called the **memory hierarchy gap** – and it is why operating systems use caching and why hardware engineers design prefetchers.

Volatile vs Non-Volatile

Type	Volatile?	Example
RAM	Yes	DRAM – data gone on power loss
Cache	Yes	SRAM

Type	Volatile?	Example
ROM	No	Read-only firmware
SSD/HDD	No	Persistent storage
NVRAM	No	UEFI variables, CMOS

Forensics insight: RAM contains secrets that vanish on power-down – encryption keys, passwords in plaintext, injected malware code. This is why memory forensics (using tools like Volatility) must happen before you power off the machine.

The Boot Sequence

- Power-on – CPU starts in a special mode
- UEFI/BIOS firmware – initializes hardware, self-tests (POST)
- Boot manager – finds the OS loader (e.g., GRUB, Windows Boot Manager)
- Kernel – decompressed into memory, initializes drivers
- Init/systemd – starts services
- Login screen – user interaction

Security insight: A **bootkit** inserts malicious code before the OS loads, hiding itself from the soon-to-run antivirus. **Secure Boot** (UEFI) verifies digital signatures of boot components using a chain of trust starting from the hardware.